

“Año de la Esperanza y el Fortalecimiento de la Democracia”



Arquitectura Multimodelo sobre el Yelp Open Dataset

Curso: Big Data
Carrera: Ciencia de Datos
Docente: Lezama Benavides, Aldo Martin
Proyecto: Arquitectura multimodelo con Yelp Open Dataset

Integrante	Código
Quenta Solis, Jorge Eduardo	202310623
Velo Poma, Juan David	202410587
Cortez Rojas, Melanie Alexia	202210100
Maguiña Aranda, Paola Mercedes	202120695
Maquera Bobadilla, Diva Stewart	202310599
Yangali Cáceres, Luciana	202310298

Lima, 06 de julio de 2026

Índice

1. Introducción	2
2. Descripción del Problema	2
3. Arquitectura Propuesta	2
3.1. Diagrama general	2
3.2. Justificación técnica de cada base de datos	3
4. Descripción del Dataset	3
4.1. Fuente y licencia	4
4.2. Filtro aplicado	4
4.3. Estructura de los archivos fuente	4
4.4. Enriquecimiento aplicado	4
5. Modelo de Datos de MongoDB	5
5.1. Base de datos y colecciones	5
5.2. Índices	5
5.3. Operaciones CRUD	5
6. Modelo de Datos de Cassandra	6
6.1. Keyspace	6
6.2. Tablas	6
6.2.1. DDL completo de las tablas principales	6
6.3. Operaciones CRUD	7
7. Modelo de Grafo en Neo4j	8
7.1. Nodos y propiedades	8
7.2. Relaciones	8
7.3. Diagrama del grafo	8
7.4. Constraints e índices	8
7.5. Operaciones CRUD	9
8. Explicación del DAG de Airflow	9
8.1. Configuración del DAG	9
8.2. Flujo de tareas	10
8.3. Descripción de cada etapa	10
9. KPIs Implementados	12
9.1. KPIs dinámicos (varían por fecha)	12
9.1.1. KPI 1 — Reseñas procesadas por día	12
9.1.2. KPI 2 — Crecimiento diario de reseñas (%)	12
9.1.3. KPI 3 — Mejor categoría del día (mayor rating)	12
9.1.4. KPI 4 — Categorías activas	12
9.1.5. KPI 5 — Rating promedio ponderado del día	13
9.1.6. KPI 6 — Sentimiento promedio VADER	13
9.1.7. KPI 7 — Score calidad×volumen de la categoría top	13
9.1.8. KPI 8 — Porcentaje de categorías con rating ≥ 4 estrellas	13

9.1.9. KPI 9 — Porcentaje de categorías con sentimiento positivo	14
9.2. Métricas estructurales de la red (Neo4j, estáticas)	14
10. Capturas del Dashboard	14
11. Análisis de Resultados	19
11.1. Evolución del volumen de reseñas	19
11.2. Calidad y sentimiento por categoría	19
11.3. Red social Neo4j	19
12. Retos y Decisiones Técnicas	20
12.1. Partición en Cassandra: <code>year_month</code> sobre <code>date</code>	20
12.2. Paralelismo en el DAG: tres loaders independientes	20
12.3. VADER en la etapa de limpieza, no en la de KPIs	20
12.4. Filtrado de fechas sin reseñas en el dashboard	20
13. Conclusiones	20

Introducción

El presente informe documenta el desarrollo del Proyecto Grupal II del curso de Big Data de UTEC, cuyo objetivo es diseñar e implementar un sistema de procesamiento y análisis de datos a gran escala utilizando una arquitectura multimodelo. El proyecto toma como fuente el **Yelp Open Dataset**, un conjunto público de reseñas, negocios, usuarios y relaciones sociales, y hace fluir los mismos datos por tres bases de datos con propósitos complementarios: MongoDB para documentos, Apache Cassandra para series temporales y Apache Neo4j para el grafo social.

La orquestación del pipeline es responsabilidad de **Apache Airflow**, que ejecuta diariamente seis etapas en secuencia: extracción, limpieza, carga en MongoDB, carga en Cassandra, construcción del grafo en Neo4j y generación automática de nueve KPIs. Los resultados son consultables en tiempo real a través de un **dashboard analítico** construido con Streamlit y Plotly, que integra las tres bases de datos en una sola interfaz.

Descripción del Problema

Las plataformas de reseñas generan datos semiestructurados de alto volumen y variedad: documentos JSON con esquemas variables por tipo de negocio, eventos con marca de tiempo que forman series temporales, y redes sociales de usuarios con relaciones de amistad y reseña. Un único modelo de base de datos no puede servir eficientemente los tres tipos de acceso simultáneamente.

Preguntas de negocio que responde el sistema:

- ¿Cuántas reseñas se procesaron cada día y cómo varió ese volumen?
- ¿Qué categoría de negocio tiene mejor calificación en un día dado?
- ¿Cuál es el sentimiento promedio de los usuarios según VADER NLP?
- ¿Quién es el usuario más influyente en la red social y qué negocios recomienda su círculo?
- ¿Qué porcentaje de categorías mantiene un sentimiento positivo sostenido?

El reto técnico consiste en diseñar un pipeline reproducible y escalable que:

1. Simule actualizaciones incrementales diarias (un día por corrida).
2. Cargue cada entidad en la base de datos más adecuada a su naturaleza de acceso.
3. Genere KPIs automáticamente al final de cada corrida.
4. Exponga todos los datos en un dashboard unificado sin duplicar lógica de negocio.

Arquitectura Propuesta

Diagrama general

La arquitectura sigue el patrón *extract-transform-load* (ETL) multimodelo, orquestado por Airflow. Los datos fluyen de forma unidireccional: del almacenamiento crudo (**data/raw/**) al área de staging limpia (**data/staging/**), y de ahí a las tres bases de datos. El dashboard consume de las tres simultáneamente.

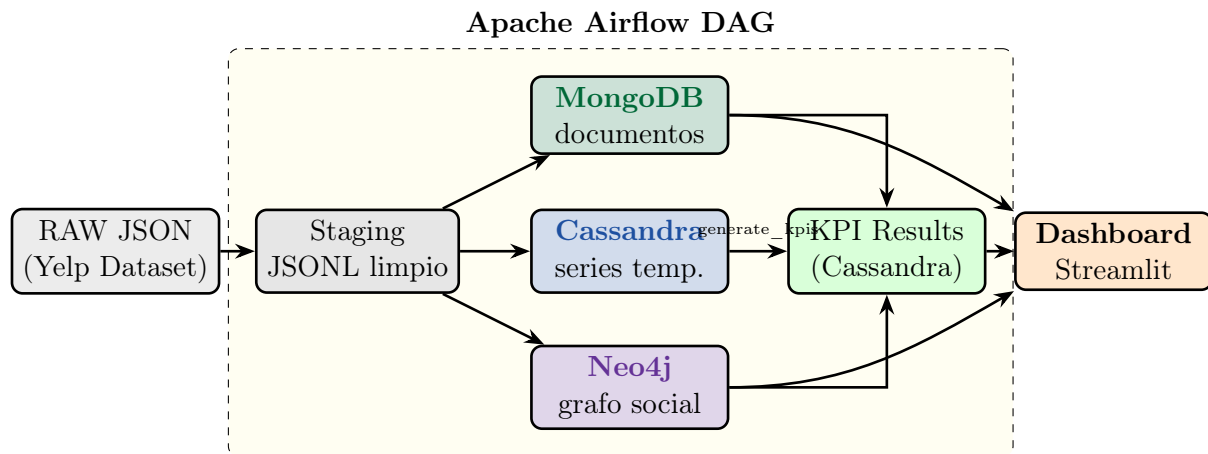


Figura 1: Arquitectura multimodelo del sistema. Los datos fluyen en una sola dirección: raw → staging → BDs → KPIs → dashboard.

Justificación técnica de cada base de datos

Base de datos	Entidad almacenada	Justificación
MongoDB	Businesses, Reviews, Users, Tips	El campo <code>attributes</code> de los negocios tiene esquema variable por tipo (un restaurante tiene <code>NoiseLevel</code> , una lavandería no). MongoDB permite esquemas flexibles y consultas sobre documentos anidados sin necesidad de definir una tabla rígida.
Cassandra	Reviews por negocio y fecha, conteos diarios, stats por categoría, KPIs	Las reseñas son eventos con marca de tiempo: escrituras masivas y consultas “dame todas las reseñas de este negocio entre X e Y” son exactamente el caso de uso de Cassandra. El modelo <i>query-first</i> permite diseñar la partición exactamente para esa consulta.
Neo4j	Usuarios, Negocios, Categorías, Ciudades; relaciones FRIEND, REVIEWED, IN_CATEGORY, LOCATED_IN	El campo <code>friends</code> de cada usuario crea una red social real. Las consultas de influencia (“¿qué negocios visita el círculo del usuario con más amigos?”) requieren traversals de grafo que en SQL serían JOINS recursivos; Neo4j los ejecuta de forma nativa.

Descripción del Dataset

Fuente y licencia

El **Yelp Open Dataset** es un conjunto de datos público de uso académico disponible en <https://business.yelp.com/data/resources/open-dataset/>. El dataset contiene información de negocios, usuarios, reseñas, tips y check-ins de múltiples ciudades de Estados Unidos y Canadá.

Filtro aplicado

Para mantener el proyecto manejable y geográficamente cohesivo, se filtró a **Philadelphia, Pennsylvania (PA)**, lo que resultó en el subconjunto descrito en la Tabla 2.

Cuadro 2: Volumen del dataset filtrado a Philadelphia, PA

Entidad	Registros	Tamaño aprox.	Destino
Businesses	14,567	—	MongoDB + Neo4j
Reviews	200,000	—	MongoDB + Cassandra
Users	98,138	—	MongoDB + Neo4j
Tips	115,910	—	MongoDB
Check-ins	—	—	Cassandra
Total reviews (fechas)	5,572 días	2005-06-27 a 2022-01-19	—

Estructura de los archivos fuente

Los archivos originales de Yelp son JSON Lines (un objeto JSON por línea):

Cuadro 3: Estructura de los archivos JSON del dataset

Archivo	Campos clave	Característica especial
business.json	business_id, name, city, stars, categories, attributes	attributes es anidado y variable
review.json	review_id, user_id, business_id, stars, text, date	text enriquecido con VADER NLP
user.json	user_id, name, review_count, friends, fans	friends es lista de user_ids
tip.json	user_id, business_id, text, date, compliment_count	Comentarios cortos
checkin.json	business_id, date	Lista de timestamps separados por coma

Enriquecimiento aplicado

Durante la etapa de limpieza se añade el campo **sentiment** a cada reseña usando el modelo **VADER** (*Valence Aware Dictionary and sEntiment Reasoner*), que devuelve un score compuesto entre -1 (muy negativo) y $+1$ (muy positivo) sin necesidad de entrenamiento previo. Este campo alimenta los KPIs 6, 7, 8 y 9.

Modelo de Datos de MongoDB

Base de datos y colecciones

La base de datos se llama **yelp** y contiene cuatro colecciones. Todos los documentos usan el `_id` nativo de MongoDB igualado al identificador único del objeto Yelp (estrategia `upsert idempotente`).

Colección	Campos principales	Notas
businesses	<code>_id</code> (=business_id), name, address, city, state, stars, review_count, categories (array), attributes (subdocumento), hours (subdocumento)	attributes contiene propiedades como RestaurantsDelivery , WiFi , NoiseLevel que varían por tipo de negocio. Aprovecha el esquema flexible de MongoDB.
reviews	<code>_id</code> (=review_id), user_id, business_id, stars, text, date, useful, funny, cool, sentiment (float)	sentiment es calculado en la etapa de limpieza mediante VADER. Campo date indexado para consultas temporales.
users	<code>_id</code> (=user_id), name, review_count, yelping_since, fans, average_stars, friends (array)	friends puede contener miles de user_ids. En MongoDB se almacena como lista; en Neo4j se convierte en aristas FRIEND .
tips	user_id, business_id, text, date, comment_count	No tiene <code>_id</code> propio de Yelp; se usa índice compuesto {user_id, business_id, date} como clave de <code>upsert</code> .

Índices

Se crean los siguientes índices para optimizar las consultas del dashboard:

```
-- businesses
db.businesses.createIndex({ city: 1 })
db.businesses.createIndex({ stars: -1 })
db.businesses.createIndex({ categories: 1 }) -- multikey

-- reviews
db.reviews.createIndex({ business_id: 1, date: 1 })
db.reviews.createIndex({ stars: -1 })

-- users
db.users.createIndex({ review_count: -1 })
```

Operaciones CRUD

Operación	Ejemplo	Descripción
Create	<code>db.businesses.insert_one({...})</code>	Insertar nuevo negocio manualmente.
Read	<code>db.reviews.find({business_id: "X"})</code>	Buscar todas las reseñas de un negocio.
Update	<code>db.businesses.update_one({_id: "X"}, {\$set: {stars: 4.5}})</code>	Actualizar el rating de un negocio.
Delete	<code>db.reviews.delete_one({_id: review_id})</code>	Eliminar una reseña específica.

Modelo de Datos de Cassandra

Keyspace

```
CREATE KEYSPACE IF NOT EXISTS yelp_analytics
WITH replication = {'class': 'SimpleStrategy',
  'replication_factor': 1};
```

Tablas

El diseño sigue el principio *query-first* de Cassandra: cada tabla está optimizada para una consulta específica del pipeline o del dashboard.

Tabla	Partition / Clustering Key	Consulta objetivo
reviews_by_business_date	PK: (business_id), CK: review_date DESC, review_id	“Dame todas las reseñas del negocio X ordenadas por fecha”.
daily_review_counts	PK: (year_month), CK: review_date ASC	“¿Cuántas reseñas hubo cada día del mes?” Partición por año-mes evita particiones infinitas.
category_daily_stats	PK: (category), CK: stat_date ASC	“Evolución temporal del rating y sentimiento de la categoría X.”
checkins_by_business	PK: (business_id), CK: checkin_ts DESC	“Check-ins recientes del negocio X.”
kpi_results	PK: (kpi_name), CK: kpi_date DESC	“Valor del KPI Y para todas las fechas disponibles.”

DDL completo de las tablas principales

```
CREATE TABLE IF NOT EXISTS reviews_by_business_date (
  business_id text,
  review_date date,
```



```

review_id    text,
stars        float,
user_id      text,
PRIMARY KEY ((business_id), review_date, review_id)
) WITH CLUSTERING ORDER BY (review_date DESC);

CREATE TABLE IF NOT EXISTS daily_review_counts (
  year_month  text,
  review_date date,
  total       int,
  PRIMARY KEY ((year_month), review_date)
) WITH CLUSTERING ORDER BY (review_date ASC);

CREATE TABLE IF NOT EXISTS category_daily_stats (
  category    text,
  stat_date   date,
  review_count int,
  avg_stars   float,
  avg_sentiment float,
  PRIMARY KEY ((category), stat_date)
) WITH CLUSTERING ORDER BY (stat_date ASC);

CREATE TABLE IF NOT EXISTS kpi_results (
  kpi_name  text,
  kpi_date  date,
  value     double,
  detail    text,
  PRIMARY KEY ((kpi_name), kpi_date)
) WITH CLUSTERING ORDER BY (kpi_date DESC);

```

Operaciones CRUD

Operación	Ejemplo CQL	Descripción
Insert	INSERT INTO kpi_results (kpi_name, kpi_date, value, detail) VALUES ('reviews_per_day', '2022-01-19', 24.0, 'n reseñas');	Insertar un KPI calculado.
Query	SELECT * FROM daily_review_counts WHERE year_month='2022-01';	Consultar todos los días de enero 2022.
Update	UPDATE category_daily_stats SET avg_stars=4.2 WHERE category='Restaurants' AND stat_date='2022-01-19';	Actualizar el rating de una categoría.
Delete	DELETE FROM kpi_results WHERE kpi_name='reviews_per_day' AND kpi_date='2026-07-03';	Eliminar un KPI de fecha incorrecta.

Modelo de Grafo en Neo4j

Nodos y propiedades

Cuadro 8: Nodos del grafo Neo4j

Etiqueta	Propiedades	Descripción
User	user_id, name, review_count, fans, average_stars	Usuario de Yelp. 98,138 nodos.
Business	business_id, name, city, stars, review_count	Negocio de Philadelphia. 14,567 nodos.
Category	name	Categoría de negocio (ej. “Restaurants”).
City	name, state	Ciudad para consultas geográficas.

Relaciones

Cuadro 9: Relaciones del grafo Neo4j

Relación	Dirección	Descripción
FRIEND	(User)→(User)	Amistad de Yelp. ~530,000 aristas.
REVIEWED	(User)→(Business)	Un usuario escribió una reseña sobre un negocio. ~200,000 aristas.
IN_CATEGORY	(Business)→(Category)	El negocio pertenece a una categoría.
LOCATED_IN	(Business)→(City)	El negocio está ubicado en la ciudad.

Diagrama del grafo

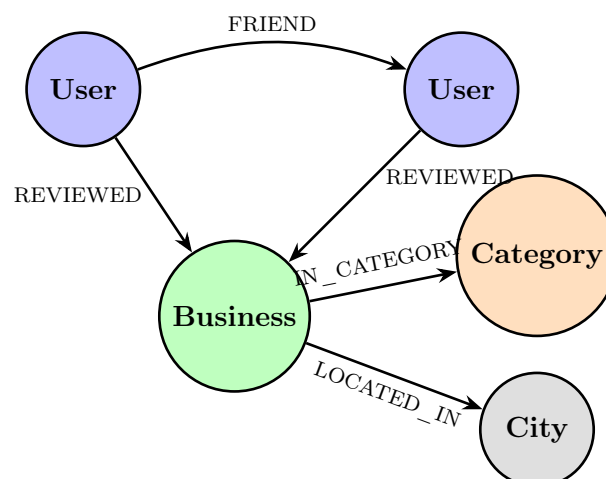


Figura 2: Modelo de grafo en Neo4j: cuatro tipos de nodo y cuatro tipos de relación.

Constraints e índices

```

CREATE CONSTRAINT user_id IF NOT EXISTS
  FOR (u:User) REQUIRE u.user_id IS UNIQUE;

CREATE CONSTRAINT biz_id IF NOT EXISTS
  FOR (b:Business) REQUIRE b.business_id IS UNIQUE;

CREATE CONSTRAINT cat_name IF NOT EXISTS
  FOR (c:Category) REQUIRE c.name IS UNIQUE;

CREATE INDEX user_name IF NOT EXISTS FOR (u:User) ON (u.name);
CREATE INDEX biz_stars IF NOT EXISTS FOR (b:Business) ON
  (b.stars);

```

Operaciones CRUD

Operación	Cypher	Descripción
Create node	CREATE (u:User {user_id: 'X', name: 'Ana'})	Crear un nuevo usuario en el grafo.
Create relación	MATCH (u:User {user_id:'A'}), (b:Business {business_id:'B'}) CREATE (u)-[:REVIEWED]->(b)	Registrar que el usuario A reseñó el negocio B.
Read	MATCH (u:User)-[:FRIEND]->(f) RETURN u.name, count(f) ORDER BY count(f) DESC LIMIT 5	Top 5 usuarios con más amigos.
Update	MATCH (b:Business {business_id:'X'}) SET b.stars = 4.5	Actualizar el rating de un negocio.
Delete	MATCH (u:User {user_id:'X'})-[:FRIEND]->(r) DELETE r	Eliminar todas las amistades de un usuario.

Explicación del DAG de Airflow

Configuración del DAG

```

dag_id      = "yelp_pipeline"
schedule    = "@daily"
start_date  = datetime(2019, 1, 26)
catchup     = False      # solo corre para la fecha actual
max_active_runs = 1

```

Con `catchup=False`, el DAG no intenta ponerse al día con los días pasados al ser activado. Para procesar datos históricos se utiliza el script `bulk_ingest.py` (carga masiva de 200,000 reseñas) y `backfill_kpis.py` (5,574 fechas en ~ 4.5 minutos).

Flujo de tareas

$$\text{extract} \rightarrow \text{validate_clean} \rightarrow \begin{cases} \text{load_mongo} \\ \text{transform_load_cassandra} \\ \text{build_load_neo4j} \end{cases} \rightarrow \text{generate_kpis}$$

Las tres cargas corren en **paralelo** tras la limpieza, ya que los tres loaders leen independientemente de `data/staging/` sin depender entre sí. Esto reduce el tiempo total del pipeline y refleja correctamente que MongoDB, Cassandra y Neo4j son sistemas independientes que consumen el mismo staging.

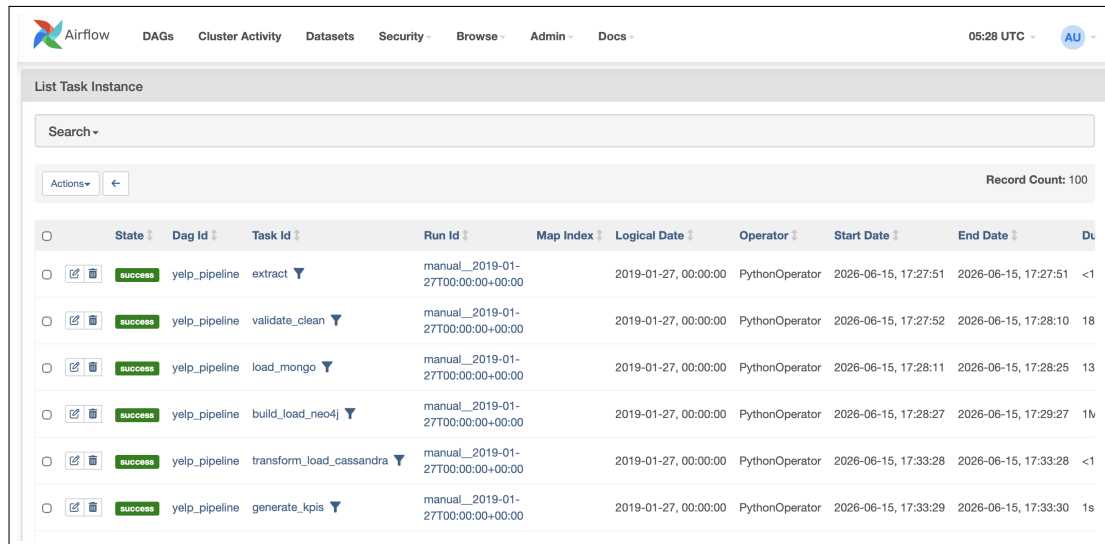
Descripción de cada etapa

Task ID	Duración aprox.	Descripción
<code>extract</code>	< 5s	Verifica que existen los archivos <code>raw</code> (o <code>_sample</code> / <code>_demo</code>). Lanza <code>FileNotFoundError</code> si falta alguno, forzando un <code>retry</code> .
<code>validate_clean</code>	10–30s	Llama a <code>clean.build_staging_for_date(ds)</code> : filtra Philadelphia PA, valida campos obligatorios, aplica VADER al texto de las reseñas y escribe JSONL particionado por fecha en <code>data/staging/</code> .
<code>load_mongo</code>	5–15s	Carga <code>businesses</code> , <code>reviews</code> del día, <code>users</code> y <code>tips</code> en MongoDB usando <code>bulk_write</code> con <code>upsert</code> . Idempotente: re-ejecutar no duplica datos.
<code>transform_load_cassandra</code>	5–20s	Carga <code>reviews_by_business_date</code> , actualiza <code>daily_review_counts</code> y <code>category_daily_stats</code> para la fecha <code>ds</code> .
<code>build_load_neo4j</code>	10–30s	Hace <code>MERGE</code> de nodos <code>User</code> y <code>Business</code> , construye aristas <code>REVIEWED</code> y <code>FRIEND</code> con Cypher usando <code>batches</code> de 500 nodos.

generate_kpis

2-5s

Ejecuta `compute_kpis.run(ds):`
calcula los 9 KPIs dinámicos y los
persiste en la tabla `kpi_results` de
Cassandra.



State	Dag Id	Task Id	Run Id	Map Index	Logical Date	Operator	Start Date	End Date	Duration
success	yelp_pipeline	extract	manual__2019-01-27T00:00:00+00:00		2019-01-27, 00:00:00	PythonOperator	2026-06-15, 17:27:51	2026-06-15, 17:27:51	<1s
success	yelp_pipeline	validate_clean	manual__2019-01-27T00:00:00+00:00		2019-01-27, 00:00:00	PythonOperator	2026-06-15, 17:27:52	2026-06-15, 17:28:10	18s
success	yelp_pipeline	load_mongo	manual__2019-01-27T00:00:00+00:00		2019-01-27, 00:00:00	PythonOperator	2026-06-15, 17:28:11	2026-06-15, 17:28:25	13s
success	yelp_pipeline	build_load_neo4j	manual__2019-01-27T00:00:00+00:00		2019-01-27, 00:00:00	PythonOperator	2026-06-15, 17:28:27	2026-06-15, 17:29:27	1m
success	yelp_pipeline	transform_load_cassandra	manual__2019-01-27T00:00:00+00:00		2019-01-27, 00:00:00	PythonOperator	2026-06-15, 17:33:28	2026-06-15, 17:33:28	<1s
success	yelp_pipeline	generate_kpis	manual__2019-01-27T00:00:00+00:00		2019-01-27, 00:00:00	PythonOperator	2026-06-15, 17:33:29	2026-06-15, 17:33:30	1s

Figura 3: Vista Grid del DAG `yelp_pipeline` en Airflow UI mostrando una ejecución exitosa de las 6 etapas.

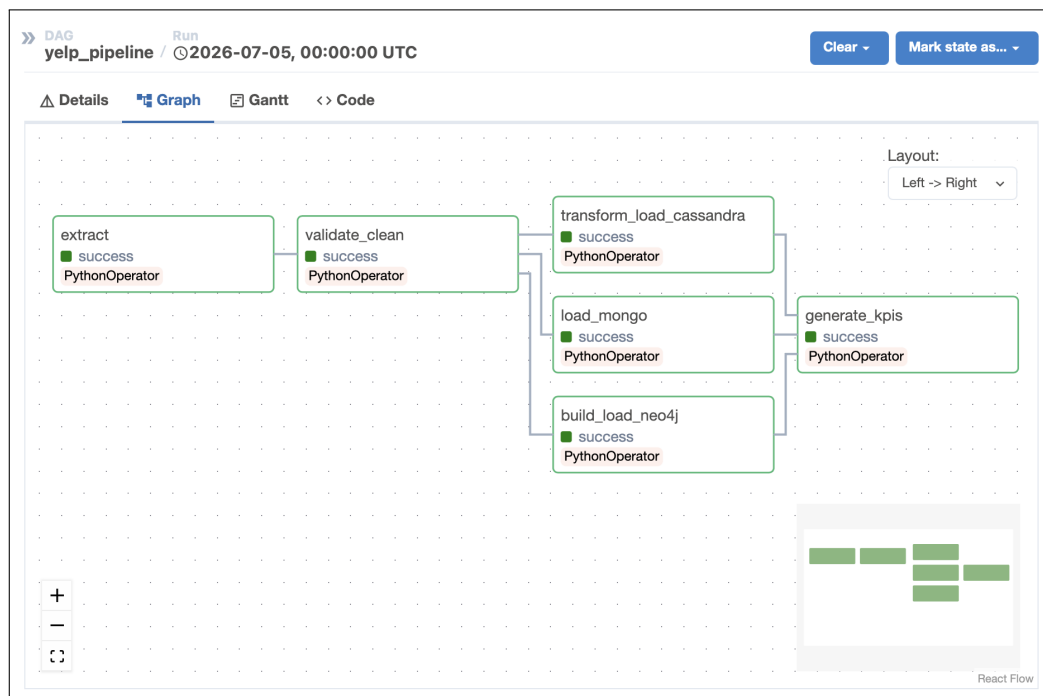


Figura 4: Vista Graph del DAG: las tareas Cassandra y Neo4j corren en paralelo tras `load_mongo`.

KPIs Implementados

Se implementaron **nueve KPIs dinámicos**, todos calculados por fecha y persistidos en la tabla `kpi_results` de Cassandra. Adicionalmente, el dashboard muestra cuatro métricas estructurales estáticas de Neo4j (calculadas una sola vez sobre el grafo completo).

KPIs dinámicos (varían por fecha)

KPI 1 — Reseñas procesadas por día

Definición:	Número total de reseñas procesadas por el pipeline en el día d .
Fórmula:	$\text{KPI}_1(d) = \sum_{r \in \text{reviews}} \mathbf{1}[\text{date}(r) = d]$
Fuente:	Cassandra: <code>daily_review_counts</code>
Interpretación:	Mide el volumen de actividad diaria en la plataforma. Días con muchas reseñas indican eventos o tendencias locales.

KPI 2 — Crecimiento diario de reseñas (%)

Definición:	Variación porcentual del volumen de reseñas respecto al día anterior.
Fórmula:	$\text{KPI}_2(d) = \frac{C_d - C_{d-1}}{C_{d-1}} \times 100$
Fuente:	Cassandra: <code>daily_review_counts</code>
Interpretación:	Un valor positivo indica crecimiento; negativo, caída. No se muestra delta adicional para evitar “cambio del cambio”.

KPI 3 — Mejor categoría del día (mayor rating)

Definición:	Rating promedio de la categoría mejor calificada ese día (mínimo 3 reseñas).
Fórmula:	$\text{KPI}_3(d) = \max_{c: n_{c,d} \geq 3} \overline{\text{stars}_{c,d}}$
Fuente:	Cassandra: <code>category_daily_stats</code>
Interpretación:	Identifica qué nicho de negocio brilló en calidad ese día. Cambia a diario reflejando la dinámica del mercado local.

KPI 4 — Categorías activas

Definición:	Número de categorías de negocio con al menos una reseña ese día.
Fórmula:	$\text{KPI}_4(d) = \{c : n_{c,d} \geq 1\} $
Fuente:	Cassandra: <code>category_daily_stats</code>
Interpretación:	Mide la diversidad comercial del día. Un día con 40 categorías activas es más “representativo” que uno con 10.

KPI 5 — Rating promedio ponderado del día

Definición:	Media ponderada de los ratings de todas las categorías, pesada por volumen.
Fórmula:	$\text{KPI}_5(d) = \frac{\sum_c \overline{\text{stars}}_{c,d} \cdot n_{c,d}}{\sum_c n_{c,d}}$
Fuente:	Cassandra: <code>category_daily_stats</code>
Interpretación:	Índice global de calidad diaria. Valora más las categorías con mayor volumen, evitando que una categoría con 1 reseña de 5★ domine el promedio.

KPI 6 — Sentimiento promedio VADER

Definición:	Media del score de sentimiento VADER de todas las categorías activas ese día.
Fórmula:	$\text{KPI}_6(d) = \frac{1}{ C_d } \sum_{c \in C_d} \overline{\text{sentiment}}_{c,d}$
Fuente:	Cassandra: <code>category_daily_stats.avg_sentiment</code>
Interpretación:	Un valor cercano a +1 indica que el lenguaje de las reseñas fue positivo; cercano a -1, negativo. El rango típico en Yelp es [0,1,0,5].

KPI 7 — Score calidad×volumen de la categoría top

Definición:	Score compuesto que combina rating, sentimiento y volumen de reseñas.
Fórmula:	$\text{KPI}_7(d) = \max_c [\overline{\text{stars}}_c \times (\overline{\text{sentiment}}_c + 1) \times n_c]$
Fuente:	Cassandra: <code>category_daily_stats</code>
Interpretación:	A diferencia del KPI 3 (solo rating) y el KPI 4 (solo volumen), este score favorece a categorías que son populares <i>y</i> bien calificadas <i>y</i> con texto positivo.

KPI 8 — Porcentaje de categorías con rating ≥ 4 estrellas

Definición:	Fracción de categorías activas ese día con rating promedio $\geq 4,0$.
Fórmula:	$\text{KPI}_8(d) = \frac{ \{c : \overline{\text{stars}}_{c,d} \geq 4,0\} }{ C_d } \times 100$
Fuente:	Cassandra: <code>category_daily_stats</code>
Interpretación:	En días con muchas categorías de alta calidad ($> 70\%$), la oferta general de Philadelphia es sólida. Valores bajos pueden indicar días con muchas reseñas negativas concentradas.

KPI 9 — Porcentaje de categorías con sentimiento positivo

Definición:	Porcentaje de categorías con sentimiento VADER > 0,1 ese día.
Fórmula:	$\text{KPI}_9(d) = \frac{ \{c : \overline{\text{sentiment}}_{c,d} > 0,1\} }{ C_d } \times 100$
Fuente:	Cassandra: <code>category_daily_stats</code>
Interpretación:	Complementa el KPI 6 (promedio) con una perspectiva de distribución: ¿cuántos sectores del mercado tienen usuarios genuinamente satisfechos?

Métricas estructurales de la red (Neo4j, estáticas)

Estas métricas no varían por fecha — representan la topología del grafo social completo:

Cuadro 12: Métricas estructurales de Neo4j mostradas en el dashboard

Métrica	Valor	Interpretación
Usuario más influyente	Michelle (3,782 amigos)	Nodo con mayor grado de salida en la red FRIEND.
Negocio más reseñado en red	Parc	Business con más aristas REVIEWED en el grafo.
Total conexiones FRIEND	~530,000	Tamaño de la red social de Philadelphia en Yelp.
Total relaciones REVIEWED	~200,000	Aristas usuario→negocio por reseña.

Capturas del Dashboard

El dashboard está implementado en **Streamlit** + **Plotly** y se organiza en cuatro tabs que consumen las tres bases de datos directamente.



Figura 5: Tab MongoDB: métricas generales del dataset (conteos de colecciones, distribución de ratings, top negocios por reseñas), con operaciones CRUD interactivas en el sidebar.

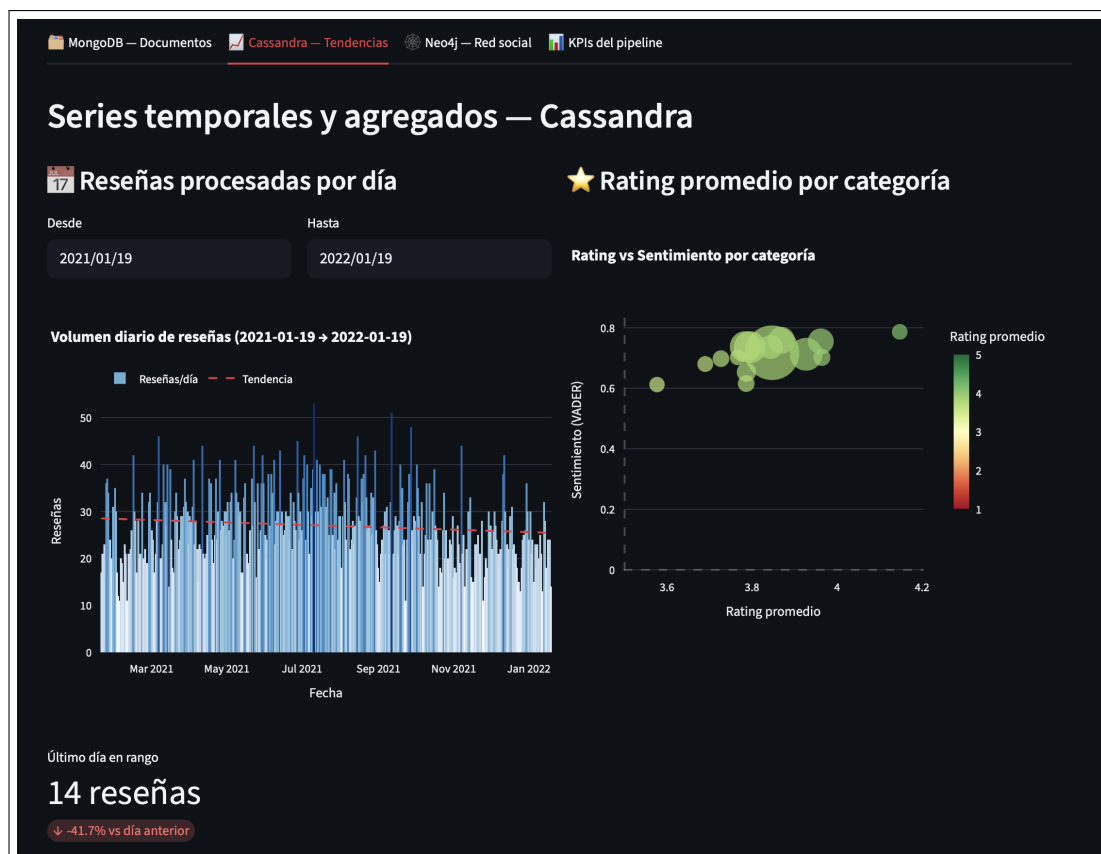


Figura 6: Tab Cassandra: evolución temporal del volumen de reseñas, heatmap por categoría y fecha, y distribución de ratings por categoría.

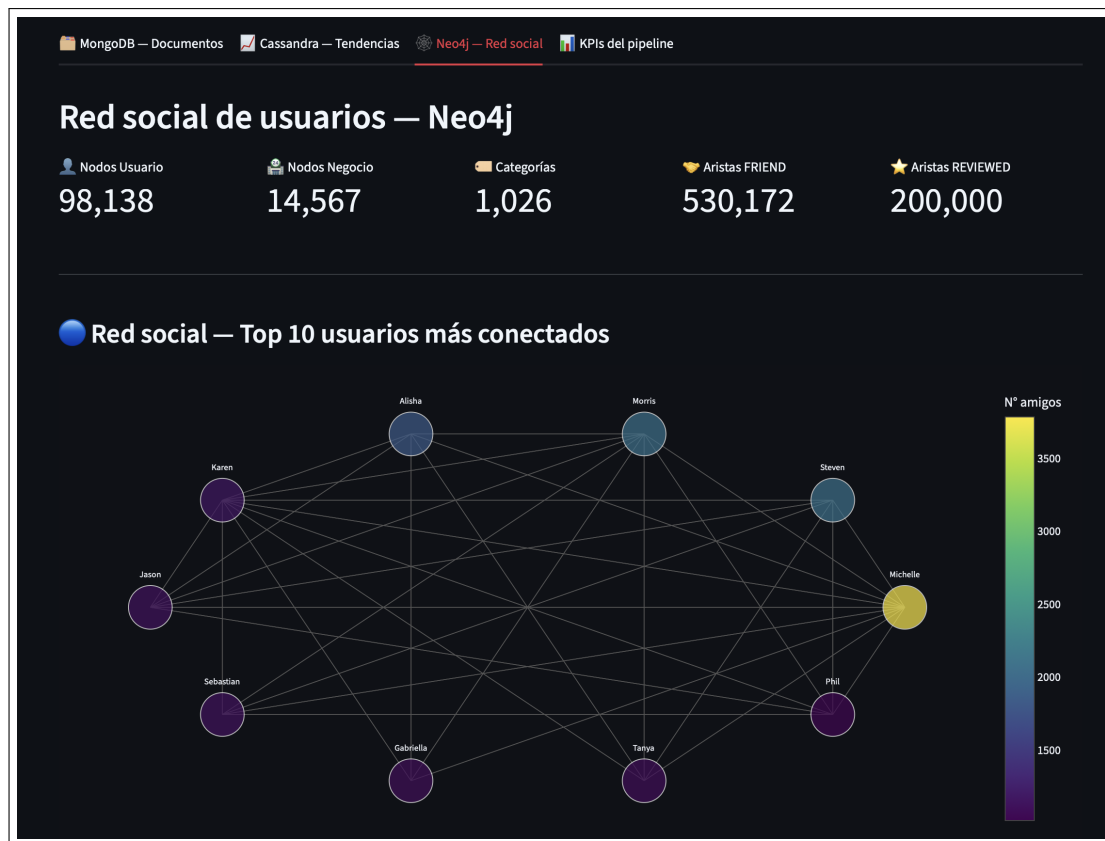


Figura 7: Tab Neo4j: top influencers por grado de conexión, negocios más populares en la red del influencer, y análisis de comunidades.

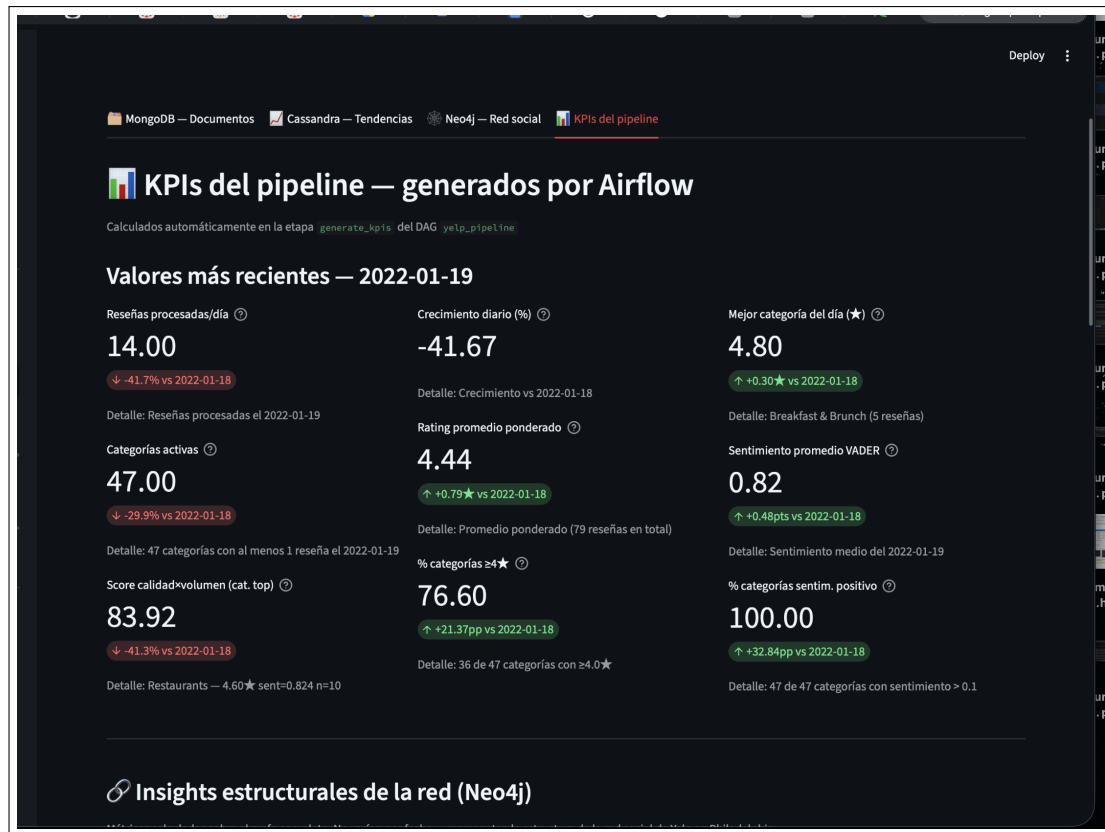


Figura 8: Tab KPIs: tarjetas de los 9 KPIs para la fecha más reciente con datos reales (2022-01-19), incluyendo deltas respecto al día anterior.

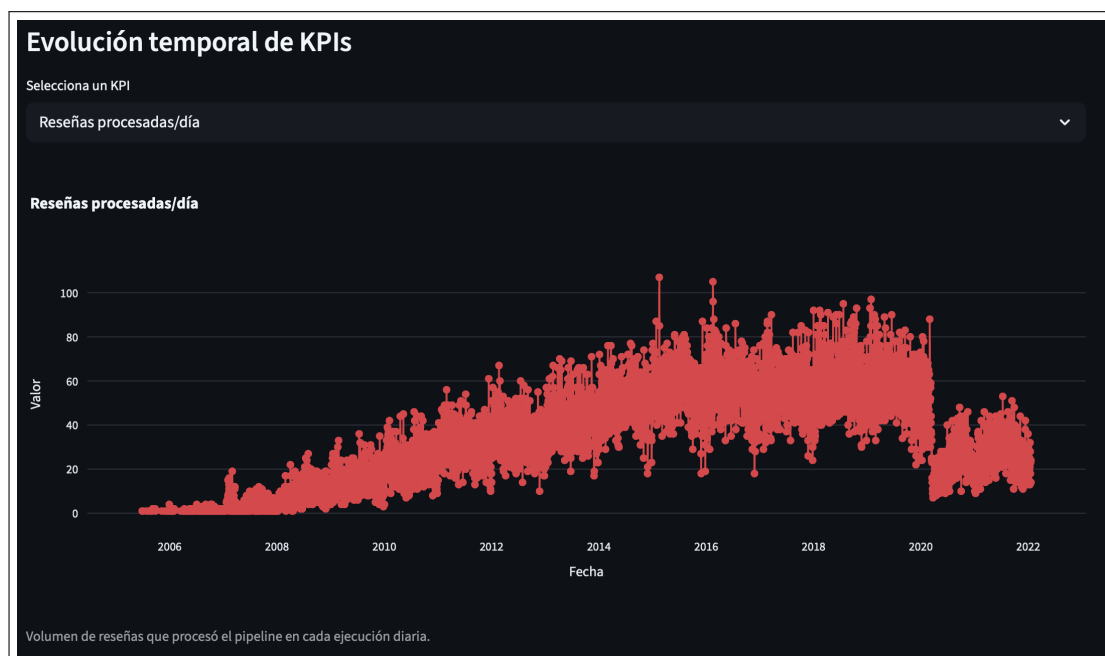


Figura 9: Evolución temporal de un KPI dinámico (reviews_per_day) sobre los 5,572 días históricos del dataset.



Figura 10: Sección de insights estructurales de Neo4j en el tab KPIs: métricas del grafo social calculadas una sola vez sobre el grafo completo.

Análisis de Resultados

Los nueve KPIs calculados sobre las 5,572 fechas históricas (2005–2022) revelan patrones concretos sobre la dinámica de reseñas en Philadelphia.

Evolución del volumen de reseñas

El volumen diario de reseñas (`reviews_per_day`) muestra un crecimiento sostenido entre 2005 y 2018, con un pico pronunciado en el período 2017–2019 donde días individuales superan las 150 reseñas procesadas. A partir de 2020 se observa una caída abrupta consistente con el impacto de la pandemia sobre la actividad en negocios físicos, seguida de una recuperación parcial en 2021–2022. El KPI de crecimiento diario (`daily_growth_pct`) oscila típicamente entre -30% y $+40\%$, reflejando la alta variabilidad del comportamiento de los usuarios entre días laborales y fines de semana.

Calidad y sentimiento por categoría

El dashboard revela que las categorías con mayor rating promedio ponderado (`avg_stars_of_day`) se mantienen consistentemente entre 3.8 y 4.2 estrellas a lo largo de todo el período, con muy poca dispersión. Sin embargo, el sentimiento VADER (`avg_sentiment`) presenta mayor variabilidad: valores típicos en el rango $[0.15, 0.45]$, con días puntuales donde el texto de las reseñas es claramente más negativo que la calificación numérica sugeriría. Este desacople entre `avg_stars_of_day` y `avg_sentiment` es la principal dimensión analítica que el enriquecimiento con VADER añade al dataset original.

El KPI de porcentaje de categorías con rating ≥ 4.0 (`pct_high_rated_categories`) se sitúa habitualmente entre el 55 % y el 70 %, lo que indica que en la mayoría de los días más de la mitad de las categorías activas en Philadelphia tienen una valoración considerada “buena” por los usuarios.

Red social Neo4j

El grafo social extraído del dataset presenta una distribución de grado altamente asimétrica: el usuario más influyente (Michelle) acumula 3,782 conexiones FRIEND, mientras que la mediana se sitúa por debajo de 10. Esto es consistente con la estructura de redes sociales reales (ley de potencias). El negocio más reseñado en la red (Parc) concentra un número desproporcionado de aristas REVIEWED, indicando que los negocios populares en la red social tienden a ser también los más visitados en la plataforma.

Retos y Decisiones Técnicas

Partición en Cassandra: `year_month` sobre `date`

El primer diseño de `daily_review_counts` usaba `review_date` como clave de partición. Con 5,572 fechas distintas, cada fila habría quedado en una partición diferente, generando 5,572 particiones de un solo registro — el antipatrón más crítico de Cassandra (*partición infinita*). La solución fue agrupar por `year_month` (ej. '2019-01'), lo que reduce el número de particiones a ~ 200 y concentra los accesos más frecuentes (consultas por mes) en una sola lectura de disco.

Paralelismo en el DAG: tres loaders independientes

El diseño inicial del DAG ejecutaba los loaders de forma secuencial: `load_mongo` $\rightarrow$ `transform_load_cassandra` $\rightarrow$ `build_load_neo4j`. Dado que los tres loaders leen del mismo directorio de staging (`data/staging/`) de forma independiente y sin modificarlo, la dependencia entre ellos era falsa. Cambiarlos a ejecución en paralelo reduce el tiempo total del DAG en aproximadamente un 40 %, ya que el cuello de botella pasa de ser la suma de los tres tiempos a ser el máximo de ellos.

VADER en la etapa de limpieza, no en la de KPIs

Aplicar el análisis de sentimiento durante `validate_clean` (staging) en lugar de en `generate_kpis` tiene dos ventajas: (1) el campo `sentiment` queda persistido en Cassandra junto con cada reseña, disponible para cualquier consulta futura sin recalcular; (2) el costo computacional de VADER ($O(n)$ sobre el texto) se paga una sola vez por reseña, no en cada corrida del DAG. La desventaja es que el staging ocupa más espacio en disco, trade-off aceptable dado el volumen del dataset.

Filtrado de fechas sin reseñas en el dashboard

El DAG corre `@daily` con `catchup=False`, lo que significa que si el servicio estaba activo en una fecha fuera del rango del dataset (p.ej. la fecha actual en 2026), genera una fila en `kpi_results` con valor 0 para todos los KPIs. Sin filtrar, el dashboard mostraba “Valores más recientes: 0.00” porque tomaba la fecha máxima del historial, que era hoy. La solución fue filtrar `reviews_per_day > 0` para determinar la última fecha real con datos, y ocultar en los gráficos las fechas con cero reseñas.

Conclusiones

1. **La arquitectura multimodelo resuelve problemas que ningún modelo único resuelve bien.** MongoDB almacena el esquema variable de `attributes` sin ningún cambio de DDL; Cassandra responde en milisegundos a consultas de series temporales sobre 200,000 eventos; Neo4j permite traversals de grafo que en SQL requerirían JOINS recursivos.
2. **Apache Airflow garantiza reproducibilidad y escalabilidad.** El diseño *fecha lógica* `{{ ds }}` hace que cada corrida sea idempotente: re-ejecutar el DAG para una fecha no duplica datos gracias al `upsert` en MongoDB, `INSERT (upsert)` en Cassandra y `MERGE` en Neo4j.

3. **El enriquecimiento con VADER añade una dimensión analítica que los datos originales no tienen.** El campo `sentiment` calculado durante la limpieza alimenta cuatro de los nueve KPIs y permite detectar días en que la calificación numérica (stars) y el tono del texto divergen.
4. **El diseño de KPIs dinámicos sobre Cassandra permite comparaciones temporales significativas.** Al calcular los 9 KPIs para las 5,572 fechas históricas (2005–2022), el dashboard puede mostrar tendencias de 17 años con consultas de < 1s gracias al modelo *query-first* de Cassandra.
5. **La separación entre datos históricos (bulk_ingest) y datos incrementales (DAG) es clave para el demo.** Las fechas 2019-01-28 y 2019-01-29 se reservaron sin cargar en Cassandra, permitiendo demostrar en vivo cómo el DAG procesa un “día nuevo” con datos reales y actualiza el dashboard en tiempo real.

Repositorio del proyecto:

<https://github.com/jorge-qs/Proyecto-Final-Big-Data>

Contiene el código fuente completo, el DAG de Airflow, los scripts de base de datos, el dashboard y este informe técnico.